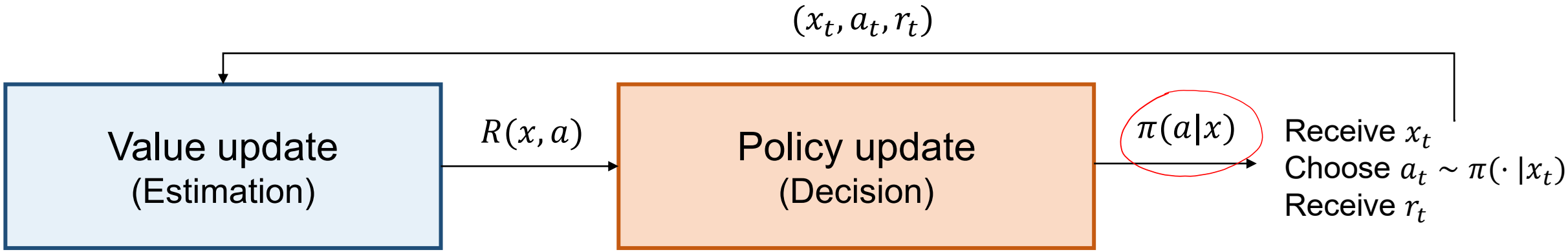
Dealing with Continuous Action Set

OpenAI SpinningUp

- https://spinningup.openai.com/en/latest/

A Unified View / Summary for Algorithms Discussed So Far

A Unified View



Contextual Bandit	Value $R(x,a)$	Policy $\pi(a\|x)$
Value-based approach (HW1)	$x \to \phi \to R_\phi(x,\cdot) \in \mathbb{R}^A$	E.G. Boltzman Induced by R_ϕ $\pi(a\|x) \propto \exp(\lambda R_\phi(x,a))$
Policy-based approach (HW2)	$\hat{r}(x,a)$ constructed from real samples (importance weighting needed)	$x \to \pi_\theta(\cdot\,\|x) \in \Delta_A$

$$\hat{r}(x,a) = \frac{r(x,a)}{\pi_0(a|x)} \mathbb{1}\{a_t = a\}$$

A Unified View

MDP	Value $Q(s,a)$	Policy $\pi(a\|s)$
Value-based approach (HW3: DQN)	$s \rightarrow \triangleright \rightarrow Q_\phi(s,\cdot) \in \mathbb{R}^A$	Induced by Q_ϕ
Policy-based approach (HW4: PPO)	$z(s,a)$ $\hat{Q}(s,a)$ constructed from 1. Pure real samples (MC estimator), or 2. Real samples + $V_\phi(s)$ (TD estimator) $s \rightarrow \triangleright \rightarrow V_\phi(s) \in \mathbb{R}$ $r+\gamma V_\phi(s')$ (importance weighting needed)	$s \rightarrow \triangleright \rightarrow \pi_\theta(\cdot\|s) \in \Delta_A$

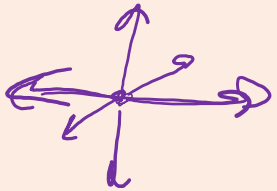
$$\hat{Q}(s,a) = \frac{\boxed{r+\gamma V_\phi(s')} - b(s)}{\pi_\theta(a|s)}$$

Continuous Contextual Bandits

Policy network + reward function approximation

Contextual Bandits with Continuous Actions

Given: Action set $\Omega \subseteq \mathbb{R}^{\cancel{d}\,3}$



(F_x, F_y, F_z)

$F_x \in [-10, 10]$

For time $t = 1, 2, \ldots, T$:

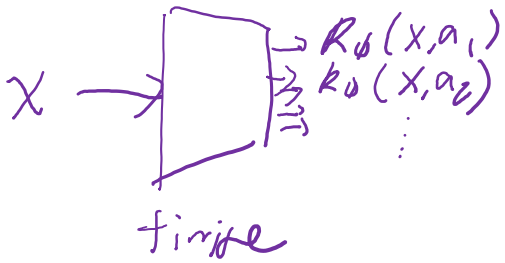
Environment reveals a context x_t

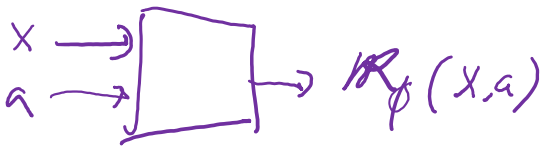
Learner chooses an action $a_t \in \Omega$

Environment reveals a reward value $r_t(x_t, a_t) = \underline{R(x_t, a_t)} + \underline{\text{noise}}$

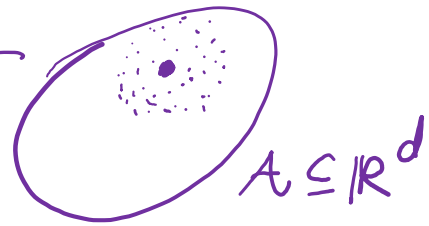
Value-Based Approach

- Use supervised learning to learn a reward function $R_\phi(x, a)$





- How to perform the exploration strategies (like ϵ-Greedy)?
  - How to find $\text{argmax}_a R_\phi(x, a)$?
  - Usually, there needs to be another **policy learning procedure** that helps to find $\text{argmax}_a R_\phi(x, a)$ $\in \mathbb{R}^d$
  - Then we can explore as $a_t = \text{argmax}_a R_\phi(x, a) + \mathcal{N}(0, \sigma^2 I)$ $\mu_\theta(x)$



$\underset{\theta}{\text{argmax}}\ R_\phi(x, \mu_\theta(x))$

Value-Based Approach

But more precisely, this is a combination of value and policy approaches

For $t = 1,2,\dots,T$:

$\mu_\theta(x) = (1, 1.5, 2)$

$a_t = (1 + N(0,\sigma^2), 1.5 + N(0,\sigma^2), 2 +)$

Receive context x_t

Take action $a_t = \mathcal{P}_\Omega\big(\mu_\theta(x_t) + \mathcal{N}(0, \sigma^2 I)\big) \in \mathbb{R}^d$

$\sigma_\theta(x_t)^2 I$

Receive $r_t(x_t, a_t)$

project in

Update the reward model:

$$\phi \leftarrow \phi - \lambda \nabla_\phi \left[\left(R_\phi(x_t, a_t) - r_t(x_t, a_t)\right)^2\right]$$

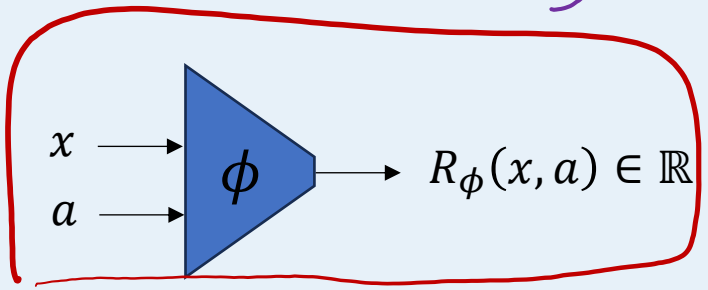
Update policy:

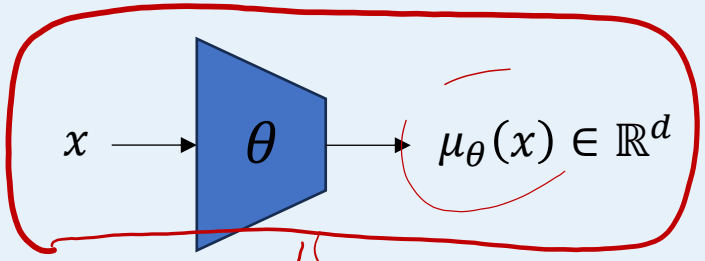
$$\theta \leftarrow \theta + \eta \nabla_\theta R_\phi(x_t, \mu_\theta(x_t))$$

$x \to \boxed{\theta} \xrightarrow{\mu_\theta(x)} \boxed{\phi} \to R_\phi(x,a)$

Equivalent policy parametrization
$\pi_\theta(\cdot\,|x) = \mathcal{N}(\mu_\theta(x), \sigma^2 I)$

Force : $[-10, 10]$
Torque : $[-0.5, 0.5]$





$\boxed{\arg\max_a R_\phi(x,a)} \approx \mu_\theta(x)$

Think of this as a continuous-action counterpart of ϵ-Greedy

Continuous Contextual Bandits

With only policy network

Gradient Ascent / PPO

For $t = 1,2,\ldots,T$:

Receive context x_t

Let $a_t = \mu_{\theta_t}(x_t) + \mathcal{N}(0, \sigma^2 I)$

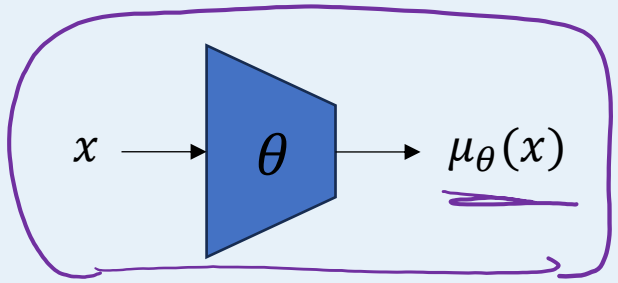
Receive $r_t(x_t, a_t)$

Update policy: (Ideally)

$$\theta_{t+1} \leftarrow \theta_t + \alpha \, \nabla_\theta \int_a \pi_\theta(a|x_t)\, R(x_t, a)\, \mathrm{d}a \Big|_{\theta=\theta_t}$$

or $$\theta_{t+1} \leftarrow \operatorname*{argmax}_\theta \int_a \pi_\theta(a|x_t)\, R(x_t, a)\, \mathrm{d}a - \frac{1}{\eta} \mathrm{KL}(\pi_\theta(\cdot\,|x_t), \pi_{\theta_t}(\cdot\,|x_t))$$

Equivalent policy parametrization
$\pi_\theta(\cdot\,|x) = \mathcal{N}(\mu_\theta(x), \sigma^2 I)$



Objective:

$$\max_\theta \int_a \pi_\theta(a|x)\, R(x,a)\, da$$

Gradient Ascent / PPO

Review: how did we do this in finite-action case?

$$\boxed{\int_a \pi_\theta(a|x_t)\, R(x_t,a)\, da}$$

$$\text{Objective}(\theta) = \sum_a \pi_\theta(a|x_t)\, R(x_t,a)$$

$$\hat{r}(x_t,a) = \frac{r(x_t,a)}{\pi_{\theta_t}(a|x_t)} \mathbb{I}\{a_t = a\}$$

$$\mathbb{E}\left[\hat{r}(x_t,a)\right] = R(x_t,a) \quad \forall a$$

$$\text{Estimated Objective}(\theta) = \sum_a \pi_\theta(a|x_t)\, \hat{r}(x_t,a)$$

$$= \boxed{\pi_\theta(a_t|x_t) \cdot \frac{r(x_t,a_t)}{\pi_{\theta_t}(a_t|x_t)}}$$

$$\mathbb{E}_{a_t \sim \pi_{\theta_t}(\cdot|x_t)}\left[\frac{r(x_t,a)}{\pi_{\theta_t}(a|x_t)} \mathbb{I}(a_t = a)\right]$$

$$= \int_{a_t} \frac{r(x_t,a)}{\pi_\theta(a|x_t)} \mathbb{I}(a_t = a)\, \pi_{\theta_t}(a_t|x_t)\, da_t$$

Gradient Ascent / PPO

$$\pi_\theta(a|x) = \frac{1}{(2\pi\sigma^2)^{d/2}}\left(-\frac{1}{2\sigma^2}\|a-\mu_\theta(x)\|^2\right)$$

For $t = 1,2,\dots,T$:

Receive context x_t

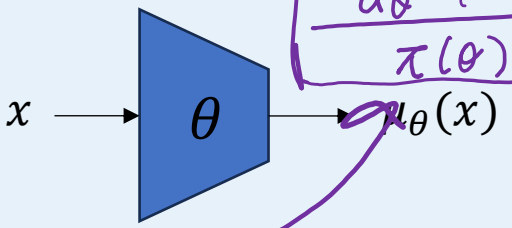
Let $a_t = \mu_{\theta_t}(x_t) + \mathcal{N}(0,\sigma^2 I)$

Receive $r_t(x_t,a_t)$

Equivalent policy parametrization
$\pi_\theta(\cdot\,|x) = \mathcal{N}(\mu_\theta(x),\sigma^2 I)$

$$\frac{\frac{d}{d\theta}\pi(\theta)}{\pi(\theta)} = \frac{d}{d\theta}\log\pi(\theta)$$

Plug this in



Update policy:

$$\theta_{t+1} \leftarrow \theta_t + \alpha\,\frac{\nabla_\theta \pi_\theta(a_t|x_t)}{\pi_{\theta_t}(a_t|x_t)}\Big|_{\theta=\theta_t}(r_t(x_t,a_t) - b_t(x_t)) = \theta_t + \alpha\nabla_\theta \log\pi_\theta(a_t|x_t)\,(r_t(x_t,a_t) - b_t(x_t))\Big|_{\theta=\theta_t}$$

PG

or

$$\theta_{t+1} \leftarrow \operatorname*{argmax}_\theta \frac{\pi_\theta(a_t|x_t)}{\pi_{\theta_t}(a_t|x_t)}\,(r_t(x_t,a_t) - b_t(x_t)) - \frac{1}{\eta}\underbrace{\mathrm{KL}(\pi_\theta(\cdot\,|x_t),\pi_{\theta_t}(\cdot\,|x_t))}_{\frac{1}{2\sigma^2}\|\mu_\theta(x_t)-\mu_{\theta_t}(x_t)\|^2}$$

(PPO)

Continuous RL

Policy network + Q function approximation

DDPG

Deep Deterministic Policy Gradient (DDPG)

For $k = 1,\ 2, \ldots$

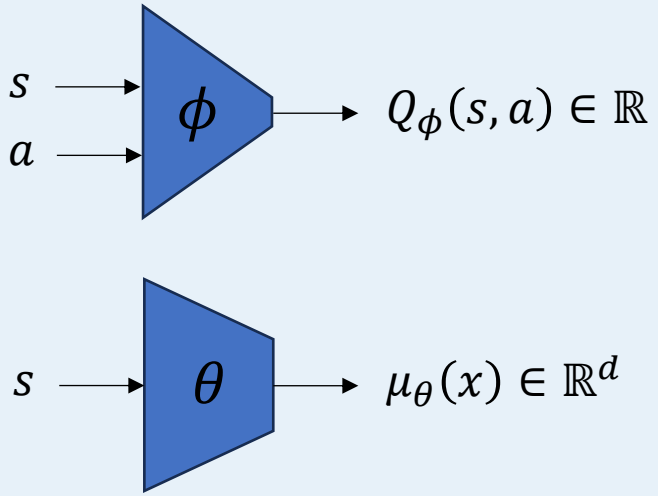
Use $\mu_\theta(s) + \mathcal{N}(0, \sigma^2)$ to collect samples and place them in **replay buffer**

Sample a batch $\{(s_i, a_i, r_i, s_i')\}_{i=1}^n$ from the replay buffer

$$\phi \leftarrow \phi - \lambda \nabla_\phi \sum_{i=1}^{n} \left(Q_\phi(s_i, a_i) - r_i - \gamma Q_{\bar{\phi}}\left(s_i', \mu_{\bar{\theta}}(s_i')\right)\right)^2$$

$$\theta \leftarrow \theta + \eta \sum_{i=1}^{n} \nabla_\theta Q_\phi\left(s_i, \mu_\theta(s_i)\right)$$

$$\bar{\phi} \leftarrow \tau\phi + (1-\tau)\bar{\phi}, \qquad \bar{\theta} \leftarrow \tau\theta + (1-\tau)\bar{\theta}$$



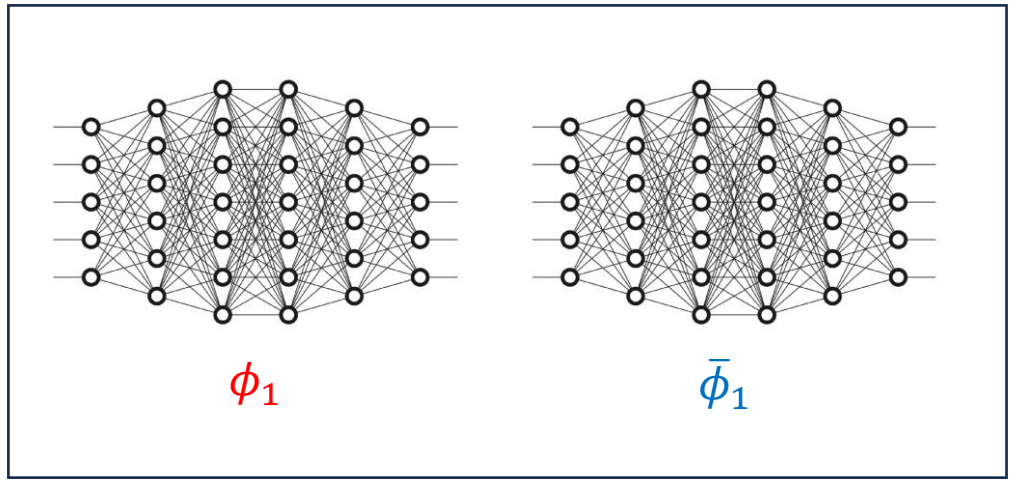
Lillicrap et al., Continuous control with deep reinforcement learning. 2015.

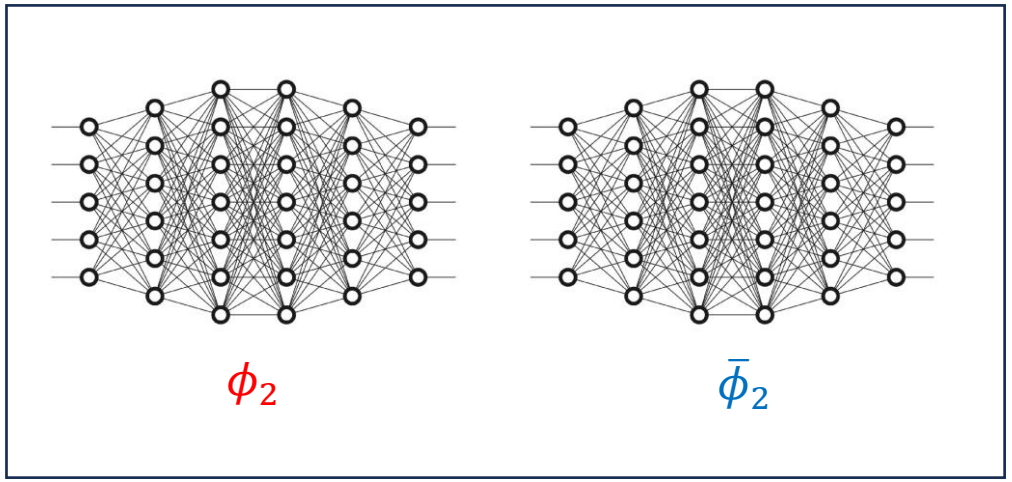
https://spinningup.openai.com/en/latest/algorithms/ddpg.html

TD3

Further Stabilizing DDPG (1/3): Twin Delayed DDPG

- Double Q-learning





Double Q-learning: When training ϕ_1, instead of using $Q_{\bar{\phi}_1}$ to evaluate the regression target, use $\cancel{Q_{\bar{\phi}_2}}$

TD3: $\min\left\{Q_{\bar{\phi}_1}, Q_{\bar{\phi}_2}\right\}$

Double Q-learning: Use independent samples to train ϕ_1 and ϕ_2

TD3: Use the same set of samples
(the independence between ϕ_1 and ϕ_2 only comes from random initialization)

Further Stabilizing DDPG (2/3): Twin Delayed DDPG

- Target policy smoothing

DDPG: use $Q_{\overline{\phi}}(s', \mu_{\overline{\theta}}(s'))$ as the regression target

TD3: sample $a' = \mu_{\overline{\theta}}(s') + \mathcal{N}(0, \sigma^2)$

use $Q_{\overline{\phi}}(s', a')$ as the regression target

Further Stabilizing DDPG (3/3): Twin Delayed DDPG

- Delayed policy updates: running multiple steps of value updates before running one step of policy update

Remark: all three changes make it harder for the policy $\mu_\theta(s)$ to exploit the error of the Q function $Q_\phi(s, a)$

Twin Delayed DDPG (TD3)

For $k = 1,\ 2, \ldots$

Use $\mu_\theta(s) + \mathcal{N}(0, \sigma^2)$ to collect samples and place them in replay buffer

Sample a batch $\{(s_i, a_i, r_i, s_i')\}_{i=1}^N$ from the replay buffer

For each sample i, draw $a_i' \sim \mu_{\bar{\theta}}(s_i') + \mathcal{N}(0, \sigma^2 I)$

$$\phi_j \leftarrow \phi_j - \lambda \nabla_{\phi_j} \sum_{i=1}^{N} \left(Q_{\phi_j}(s_i, a_i) - r_i - \gamma \min_{\ell=1,2} Q_{\bar{\phi}_\ell}(s_i', a_i') \right)^2 \qquad \forall j = 1,2$$

If $k \bmod K = 0$:

$$\theta \leftarrow \theta + \eta \sum_{i=1}^{N} \nabla_\theta Q_{\phi_1}(s_i, \mu_\theta(s_i))$$

$$\bar{\theta} \leftarrow \tau\theta + (1-\tau)\bar{\theta}$$

$$\bar{\phi}_j \leftarrow \tau\phi_j + (1-\tau)\bar{\phi}_j \qquad \forall j = 1,2$$

Fujimoto et al., Addressing Function Approximation Error in Actor-Critic Methods. 2018.
https://spinningup.openai.com/en/latest/algorithms/td3.html

Continuous RL

Policy network + MC or TD estimator

PPO

Same pseudocode as in the finite-action case

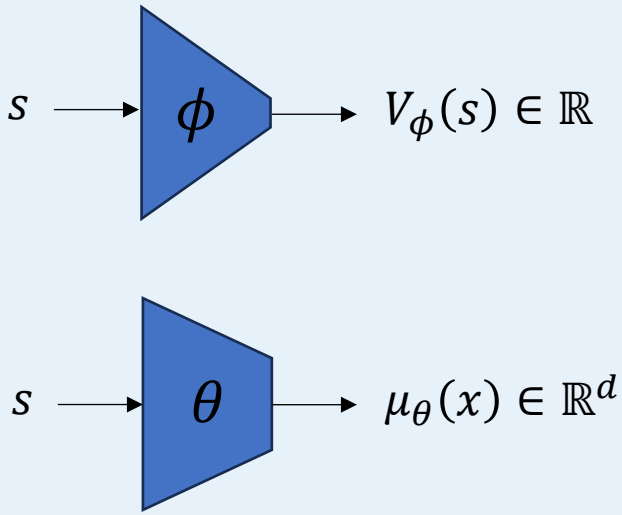
For $k = 1,\ 2, \ldots$

For $i = 1, 2, \ldots, N$:

Choose action $a_i \sim \pi_{\theta_k}(\cdot\,|s_i) = \mu_{\theta_k}(s_i) + \mathcal{N}(0, \sigma^2 I)$

Receive reward $r_i \sim R(s_i, a_i)$ and $s_i' \sim P(\cdot\,|s_i, a_i)$

$s_{i+1} = s_i'$ if episode continues, $s_{i+1} \sim \rho$ if episode ends



For $j = 1, 2, \ldots, M$:

$$\theta \leftarrow \theta + \eta \nabla_\theta \frac{1}{N} \sum_{i=1}^{N} \left(\frac{\pi_\theta(a_i|s_i)}{\pi_{\theta_k}(a_i|s_i)} (\boldsymbol{G_i} - V_\phi(s_i)) - \frac{1}{\eta} \mathrm{KL}\big(\pi_\theta(\cdot\,|s_i), \pi_{\theta_k}(\cdot\,|s_i)\big) \right)$$

$$\phi \leftarrow \phi - \alpha \nabla_\phi \frac{1}{N} \sum_{i=1}^{N} \left(V_\phi(s_i) - r_i - \gamma V_{\phi_k}(s_{i+1}) \right)^2$$

$\theta_k \leftarrow \theta, \qquad \phi_k \leftarrow \phi$

Remark

- We developed a **unified view** for value-based and policy-based algorithms.
- We have extended the ideas for finite-action case to continuous-action case
  - The original “policy-based” algorithms have the same pseudocode, except that policies are represented differently. These methods usually involve training $\pi_\theta(a|s)$ and $V_\phi(s)$
  - For the original “value-based” algorithms, we add a policy network to approximate the argmax operation. These methods usually involve training $\pi_\theta(a|s)$ and $Q_\phi(s,a)$